PrediQL: Automated Testing of GraphQL APIs with LLMs

Shaolun Liu[1*], Sina Marefat[2*], Omar Tsai[1], Yu Chen[1],
Zecheng Deng[1], Jia Wang[1], Mohammad A. Tayebi[1]
[1] Simon Fraser University, Canada
[2] K. N. Toosi University of Technology, Iran
shaolun.liu@sfu.ca,sina.marefat@email.kntu.ac.ir,omar@ztasecurity.com,
yca518@sfu.ca,zda35@sfu.ca,jwa454@sfu.ca,tayebi@sfu.ca

Abstract

GraphQL's flexible query model and nested data dependencies expose APIs to complex, context-dependent vulnerabilities that are difficult to uncover using conventional testing tools. Existing fuzzers either rely on random payload generation or rigid mutation heuristics, failing to adapt to the dynamic structures of GraphQL schemas and responses. We present PrediQL, the first retrieval-augmented, LLM-guided fuzzer for GraphQL APIs. PrediQL combines large language model reasoning with adaptive feedback loops to generate semantically valid and diverse queries. It models the choice of fuzzing strategy as a multi-armed bandit problem, balancing exploration of new query structures with exploitation of past successes. To enhance efficiency, PrediQL retrieves and reuses execution traces, schema fragments, and prior errors, enabling self-correction and progressive learning across test iterations. Beyond input generation, PrediQL integrates a context-aware vulnerability detector that uses LLM reasoning to analyze responses, interpreting data values, error messages, and status codes to identify issues such as injection flaws, access-control bypasses, and information disclosure. Our evaluation across open-source and benchmark GraphQL APIs shows that PrediQL achieves significantly higher coverage and vulnerability discovery rates compared to state-of-the-art baselines. These results demonstrate that combining retrieval-augmented reasoning with adaptive fuzzing can transform API security testing from reactive enumeration to intelligent exploration.

Keywords

GraphQL Security, API Fuzzing, Large Language Models, Retrieval-Augmented Generation, Multi-Armed Bandit Learning

1 INTRODUCTION

Modern software systems are often built from many independent microservices that communicate through well-defined APIs. Among contemporary API styles, GraphQL has gained significant adoption because it allows clients to request precisely the data they need through a single flexible endpoint. This design improves efficiency and mitigates the problem of over-fetching, common in REST [8, 16] or gRPC [8] APIs.

According to a 2024 industry survey, approximately 61% of respondents reported using GraphQL in production, and about 10 % indicated that they were replacing REST with GraphQL in their systems [34, 49]. However, this widespread adoption has also exposed security weaknesses in many deployments. A recent study reported that roughly 69 % of the scanned public GraphQL API services suffered from unrestricted resource consumption vulnerabilities, making them susceptible to denial of service (DoS) attacks via deep-nested or costly queries [25, 27, 35]. These issues highlight that GraphQL introduces unique security challenges, such as unbounded query depth, schema exposure, injection in nested arguments, and inconsistent access control across linked queries and mutations [7]. Consequently, effective testing of GraphQL APIs requires more than traditional input fuzzing; it must incorporate a contextual understanding of schema relationships and multistep interactions.

Existing testing tools fall into two main groups: black-box fuzzers and schema-aware fuzzers. Black-box fuzzers explore input space randomly, but do not respect GraphQL structure. Schema-aware tools, such as EvoMaster [17–19], improve test coverage by using the schema; but rely on simple template mutations and ignore cross-field or cross-operation dependencies. Recent systems such as GraphQLer [50] take an important step by analyzing producer–consumer relationships between queries and mutations, discovering vulnerabilities missed by older tools. However, even these approaches remain static; their generation logic does not adapt to execution feedback or changing context. In short, current GraphQL fuzzers cannot yet reason adaptively about API behavior.

Recent advances in large language models (LLMs) offer a new direction on how to build intelligent, feedback-driven fuzzers. LLMs can reason about structured input formats, infer valid parameters from schema fragments, and generate meaningful queries [43, 46]. When combined with retrieval mechanisms that use past traces, schema segments, or error logs [37, 41, 48], the fuzzer can refine future inputs based on what it has already learned. Recent LLM-assisted fuzzing solutions [22, 33, 36, 45, 51, 52] show that generative reasoning improves coverage in binary and protocol fuzzing. Yet, no prior research has applied LLM-based fuzzing to GraphQL, whose nested queries and dependency-rich structure require contextual reasoning and adaptive feedback. Work such as WENDIGO [44] targets denial-of-service discovery in GraphQL using deep reinforcement learning, and Perera et al. [47] explore detecting malicious GraphQL queries with LLMs and neural classifiers, but neither focuses on adaptive fuzzing.

To close this gap, we present PrediQL, an automated GraphQL testing framework that joins schema introspection, retrieval-augmented LLM prompting [41], multi-armed bandit learning [15, 20, 40] and self-correction into a single feedback loop. PrediQL treats the LLM as a guided component rather than an oracle. It first extracts schema information through introspection and then retrieves relevant examples and past errors to generate the query in real feedback. A bandit-based selector dynamically

*Equal contribution.

chooses between different prompting strategies, balancing exploration and exploitation. This design allows PrediQL to generate valid, diverse, and context-sensitive queries that explore deeper parts of the schema and reveal complex vulnerabilities.

We evaluated PrediQL using multiple large language models across a diverse set of open-source and benchmark GraphQL APIs. The results show that PrediQL consistently improves test coverage, by an average of 16% with a maximum improvement of 50%, and discovers more context-dependent vulnerabilities compared to established baselines. Among the tested configurations, the `GPT-5 Mini` achieved the highest coverage, while smaller models like Llama-3-8B offered competitive results with reduced computational cost. In addition to broader coverage, PrediQL demonstrates stronger vulnerability detection capabilities, accurately identifying injection flaws, access control bypasses, and information disclosure cases that existing tools often miss.

In summary, this paper makes three main contributions: (1) We present PrediQL, the first retrieval-augmented, LLM-guided GraphQL fuzzer with adaptive strategy selection, modeled as a multi-armed bandit problem [15, 20, 40] to improve efficiency and reduce redundant requests. (2) We design a context-aware vulnerability detector that leverages LLM reasoning to interpret responses and identify diverse vulnerability categories beyond static rule-based detection. (3) We conduct extensive experimental evaluation on open-source and benchmark GraphQL APIs, demonstrating that PrediQL achieves higher coverage and detects more context-dependent vulnerabilities than existing tools.

2 BACKGROUND & RELATED WORK

Modern web applications increasingly adopt GraphQL for its flexible and efficient data fetching model. While this paradigm simplifies client–server interaction, it also introduces a new class of security challenges that differ from those found in traditional REST APIs. In this section, we first review the fundamentals of GraphQL, then summarize its known vulnerability classes, and finally discuss prior work on API testing, GraphQL security analysis, large language model (LLM)–assisted fuzzing, prompt engineering, and adaptive test strategy selection.

2.1 GraphQL Fundamentals

GraphQL provides a structured and flexible alternative to traditional REST APIs, offering three key features [7, 8, 16]:

- *Data as a graph:* GraphQL organizes data as a graph of interconnected objects, allowing clients to retrieve all required information in a single request and mitigating both under-fetching and over-fetching.
- *Strong typing:* Each GraphQL API exposes a schema that defines data types (objects) and operations (queries and mutations). This strong type system ensures predictable responses, simplifies error handling, and requires clients to explicitly specify which fields to return.
- *Single endpoint.* All requests are processed through a unified endpoint, providing a consistent interface for both data retrieval and modification.

A GraphQL schema defines two primary categories of data types: *scalars* and *objects*. Scalars represent atomic values such as `Int`, `Float`, `String`, `Boolean`, and `ID`. Objects, on the other hand, are user-defined entities composed of multiple fields, which may themselves be scalars, objects, or lists of other types. This nested composition creates a rich graph structure that captures relationships among entities and allows clients to traverse and query linked data seamlessly.

Clients interact with GraphQL APIs through two main operation types:

- *Queries* retrieve data from the server, functioning similarly to HTTP `GET` requests in REST APIs. A special form, the *introspection query*, allows clients to inspect the schema and obtain metadata about available types and operations.
- *Mutations* modify data on the server, corresponding to creation, update, or deletion actions.

Both queries and mutations enable fine-grained field selection, ensuring clients request exactly the data they need. Each field may also take arguments, allowing deeply nested and parameterized requests. A GraphQL operation is first parsed and validated against the schema, then executed by resolving each field independently. This layered execution model, together with features like schema introspection, makes GraphQL powerful but also complex to test securely.

2.2 GraphQL Vulnerabilities

GraphQL provides powerful ways to query and manage data, but its flexibility also brings new types of security risks. Some of these problems are similar to common web application issues, such as broken access control, injection flaws, and misconfigurations, while others are specific to how GraphQL is designed. These GraphQL-specific issues expand the attack surface and need special attention. We group them into three main categories:

- *Query Abuse Vulnerabilities.* Because GraphQL allows users to send highly flexible queries, attackers can take advantage of this feature to overload or explore the system. A common example is misuse of the *introspection query*, which reveals the entire schema, including all types, fields, and relationships. This information helps attackers craft more targeted and harmful queries. GraphQL is also exposed to *Denial of Service (DoS)* attacks caused by deeply nested or repetitive queries that consume large amounts of server resources, leading to slowdowns or outages.
- *Injection Vulnerabilities.* GraphQL inputs can be an entry point for injection attacks if they are not properly checked or sanitized. The most serious case is *SQL Injection*, where unsafe user inputs are added to database queries, allowing data theft or manipulation. Other examples include *Path Injection* and *Cross-Site Scripting (XSS)*. These can result in stolen sessions, leaked data, or malicious code running in the browser. Weak input handling in GraphQL therefore puts both the backend and users at risk.
- *Access Control Vulnerabilities.* Access control problems occur when a GraphQL API fails to properly restrict what data users can access. A typical example is *Insecure Direct Object Reference (IDOR)*, where attackers change object identifiers to view or modify restricted data. Another example is *batched attacks*, where multiple operations are combined in one request to bypass individual security checks. Fixing such issues is difficult because

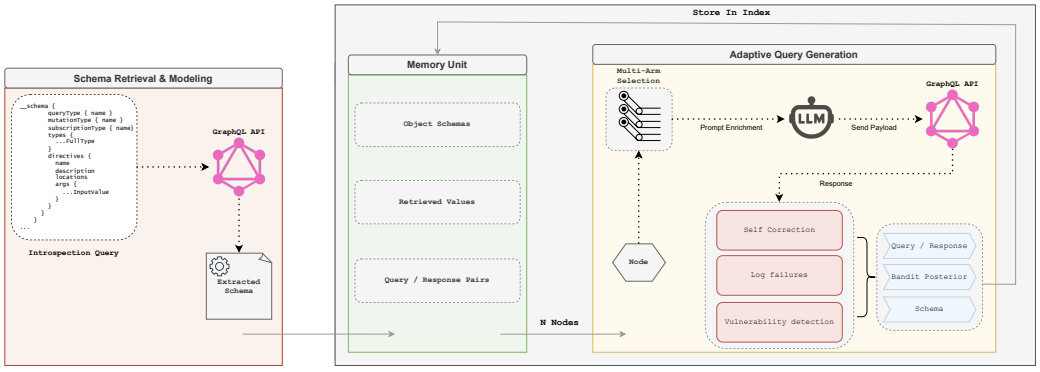

Figure 1: workflow of PrediQL.

GraphQL schemas often include many linked fields and relationships. Testing for these problems requires context-aware and dependency-based approaches that can understand how queries, mutations, and objects interact, helping to uncover access control flaws that traditional testing might miss.

2.3 Related Work

GraphQL Security Testing. Security testing for GraphQL has evolved from basic black-box approaches to more sophisticated schema-aware methods. Early community tools such as GraphQL-Cop [26], GraphCrawler [31], CrackQL [24], and Schemathesis [32] rely on introspection to generate single-request mutations but lack systematic reasoning about dependencies between queries and mutations. EvoMaster extends evolutionary and random testing to GraphQL APIs, supporting both black-box and white-box modes to broaden coverage [17, 18]. Industry-standard scanners such as OWASP ZAP [14] and BurpSuite [1] include GraphQL modules (e.g., Burp's Auto GQL Scanner) but primarily perform generic payload fuzzing and introspection-based attacks rather than exploring multi-step workflows. GraphQLer [50] introduced the first context-aware GraphQL security testing framework. It analyzes the schema to infer producer–consumer dependencies among queries and mutations, constructs a dependency graph, and generates realistic chained payloads. This approach improves schema coverage and discovers previously unknown vulnerabilities compared with earlier fuzzers, motivating the need for dependency reasoning in GraphQL security testing.

LLM-Assisted Fuzzing. In dynamic analysis, fuzzing remains one of the most effective ways to automatically find software bugs. Coverage-guided fuzzers such as AFL++ [28] and libFuzzer [42], along with large-scale efforts like OSS-Fuzz [30], have discovered thousands of vulnerabilities. However, these approaches often struggle with highly structured or constrained inputs and with reaching deep execution paths. To address these challenges, recent work combines LLMs with fuzzing and related dynamic techniques. Fuzz4All [51] uses LLMs for input generation and mutation across multiple formats, maintaining diversity through an autoprompting loop and achieving higher coverage than traditional fuzzers. EL-Fuzz [21] enhances generation-based fuzzers through LLM-driven synthesis over input spaces, achieving higher coverage and revealing real-world bugs. Recent studies have explored machine learning–based detection of malicious GraphQL queries. For instance, one line of work applies deep reinforcement learning to identify denial-of-service patterns in GraphQL APIs [44], while another investigates the use of large language models, sentence transformers, and convolutional neural networks for malicious query detection [47]. There has also been work to use LLMs in text-to-GraphQL queries [39]; however, the model does not target specific API schemas. To the best of our knowledge, no prior work has applied LLM-assisted fuzzing to GraphQL, whose schema-rich, multi-operation design introduces unique challenges. PrediQL is the first to address this gap by leveraging LLM reasoning to guide test generation and vulnerability detection in GraphQL APIs.

3 METHODOLOGY

PrediQL is a modular LLM-driven GraphQL fuzzing framework organized around a closed-loop pipeline (Figure 1). At its core, PrediQL integrates an LLM-based query generator with schema introspection, semantic retrieval, adaptive arm selection, and self-correction modules. This architecture constrains the model's generation space, maintaining validity and fuzzing relevance while adapting to feedback dynamically. The key contribution lies in coupling schema-aware reasoning with adaptive prompting to produce valid, diverse, and feedback-driven queries.

Rather than relying on static mutation operators, PrediQL refines prompts with schema fragments, retrieved traces, and error signals, making each generation guided by prior results and schema insights. The system runs as a closed-loop cycle: schema modeling constrains the search space, the bandit-based selector chooses a prompting strategy, retrieval grounds prompts in real execution history, and self-correction incorporates prior errors. Prompt construction then assembles these elements into an evidence-gated LLM input; executed queries update the schema, memory, and bandit posteriors, completing the loop and allowing iterative refinement of validity, diversity, and coverage.

3.1 Schema Modeling

The PrediQL pipeline begins with a standard GraphQL introspection query against the target API. Introspection reveals the complete

schema, which includes queries, mutations, input parameters, and return types. PrediQL parses this output into a structured intermediate representation that organizes operations, argument specifications, and object definitions. To facilitate reuse and downstream automation, the schema is serialized into lightweight YAML files, separating queries, mutations, and type definitions. This schema map provides a blueprint of the API before any fuzzing or query generation takes place.

Unlike prior tools that treat introspection results as flat listings, PrediQL recursively follows links between objects to capture nested and cross-referenced types. This yields a graph-structured view of responses, enabling the system to reason about both top-level operations and the shape of deeply nested return values. Maintaining this normalized schema representation allows PrediQL to generate queries that are syntactically valid, semantically consistent, and structurally diverse, while avoiding common issues such as type mismatches or missing arguments.

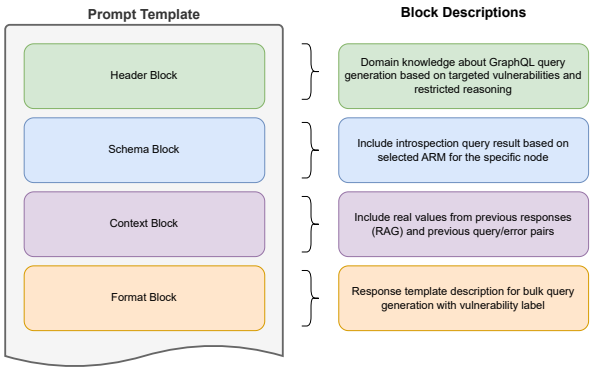

Figure 2: structure of curated prompt and enrichment sections.

3.2 Adaptive Query Generation

Adaptive Arm Selection. Query generation is framed as a multi-armed bandit problem [40], where each arm represents a distinct prompting strategy defined by four key parameters. The *Schema* parameter determines whether the full GraphQL schema is included in the LLM prompt to guide query construction. The *Arg Mode* parameter controls how arguments are generated: *known* reuses previously successful parameter values from the RAG context, *real* synthesizes realistic type-appropriate literals, and *nulls* tests optional fields with null values. The *Depth* parameter limits the nesting level of GraphQL selections to balance complexity and validity, while the *top-k* parameter specifies how many similar examples from the RAG system are retrieved as contextual grounding (see Section 4.1 for configuration details).

Since the effectiveness of each strategy depends on the endpoint's evolving behavior, the bandit formulation enables PrediQL to dynamically allocate preference toward high-performing arms while still exploring alternatives. To balance exploration and exploitation, PrediQL employs Thompson Sampling [15], rewarding arms only when generated queries both *(i)* return HTTP 200 responses and *(ii)* expand coverage. To account for non-stationary environments, rewards are exponentially discounted so that outdated strategies are gradually down-weighted. This adaptive mechanism prevents over-commitment to a single strategy while progressively amplifying those that yield meaningful coverage.

Retrieval-Augmented Generation. To mitigate the problem of hallucinated or invalid query values, PrediQL integrates retrieval-augmented generation (RAG) [41]. All prior queries and responses are embedded into a FAISS index [23]. During generation, PrediQL retrieves the top-k (where k is equal to 0, 3, or 5, depending on the selected ARM) most semantically relevant traces and injects them into the prompt.

This retrieval memory grounds the LLM in real execution history, improving syntactic fidelity, reducing repetition, and promoting diversity by surfacing alternative query structures.

Prompt Engineering. Prompt design is key to guiding query generation. PrediQL adopts four design goals: prompts must be evidence-gated, deterministic, schema-constrained, and context-aware. At runtime, prompt P is automatically assembled from five modular components: $P = [B \parallel S \parallel R \parallel E \parallel D]$:

- B: Basic restricting header (evidence-gating).
- S: Schema fragments from introspection
- R: Retrieved execution examples (RAG)
- E: Prior error–query pairs (self-correction)
- D: Strategy-specific directives (bandit arm)

This modular structure, illustrated in Figure 2, ensures that exploration happens primarily through D (arm choice), while S, R, and E stabilize performance across strategies.

3.3 Execution and Feedback

Self Correction. Failed queries are not discarded. Instead, PrediQL explicitly records schema errors and associates them with the queries that caused them. In subsequent generations, these error–query pairs are injected into the prompt as corrective signals.

This error-guided refinement turns schema violations into supervision, steering the model away from repeated mistakes and accelerating convergence toward valid, schema-compliant queries.

Context-Aware Vulnerability Detection. To assess security impact, PrediQL uses an LLM to analyze responses in context, rather than relying on predefined signatures or static rules. Each response, together with its execution metadata such as status codes and error messages, is transformed into a structured analysis prompt that directs the LLM to identify evidence of vulnerabilities, including injection flaws, access-control bypasses, and information disclosure. The resulting analyses are parsed into JSON records encoding vulnerability type, severity, confidence, and evidence. These records are stored and aggregated, yielding both granular findings and system-level trends. By conditioning on schema knowledge and execution context, PrediQL generalizes across diverse GraphQL APIs and uncovers subtle, context-dependent flaws that rule-based detectors overlook.

Table 1: GraphQL APIs used in baseline testing.

API	#Queries	#Mutations	#Objects
`UserWallet` [13]	11	15	5
`Countries` [2]	6	0	5
`Rick&Morty` [11]	9	0	7
`GraphQLZero` [9]	13	19	18
`EHRI` [4]	19	0	46
`TCGDex` [12]	6	0	12

3.4 Closed-Loop Integration

Together, these phases form a closed-loop fuzzing cycle in which schema knowledge constrains the LLM, adaptive generation explores diverse query strategies, and execution feedback continuously refines both prompts and strategy selection. This iterative design enables PrediQL to expand coverage systematically while maintaining robustness and precision in vulnerability discovery.

4 EVALUATION

We implement the design and experiments in an open-source repository*. Our experiment is structured to investigate the following research questions: **RQ1.** How can LLMs be guided to synthesize valid yet adversarial GraphQL queries that systematically expand schema coverage compared to schema-only or random fuzzing approaches? **RQ2.** How does prompt engineering and context enrichment contribute to improving schema coverage? **RQ3.** To what extent can this pipeline discover meaningful GraphQL vulnerabilities over existing rule-based methods?

4.1 Experimental Setup

API Selection. In our experimental evaluation, we employed a diverse spectrum of APIs, including open-source and openly hosted options. For open-source APIs, we obtained the backend code, established a self-hosted GraphQL server, and conducted tests. For openly hosted APIs, we utilized free-to-use reference APIs accessible on GitHub and conducted direct testing. The selected APIs are shown in Table 1.

Baselines. We evaluate PrediQL against four representative baselines that include both general-purpose and GraphQL-specific testing frameworks.

- ◇ EvoMaster. The only prior academic framework supporting GraphQL testing in both white-box and black-box modes. We use its *black-box* configuration for fair comparison. EvoMaster generates and sends GraphQL queries and mutations automatically, but employs evolutionary heuristics to mutate payloads dynamically and explore a wider response space [17–19].
- ◇ ZAP. An open source black-box vulnerability scanner maintained by the Open Web Application Security Project (OWASP) [14]. Although originally designed for traditional web applications, it includes a module for GraphQL testing that performs introspection-based payload generation and common attack simulations.
- ◇ BurpSuite. A widely used commercial web security platform developed by PortSwigger [1]. For GraphQL testing, we enable the *Auto GQL Scanner* extension [29], which automatically identifies GraphQL endpoints and injects predefined payloads to detect vulnerabilities such as injection and schema disclosure.
- ◇ GraphQLer. A recent context-aware GraphQL security testing framework [50]. It constructs a dependency graph that captures producer–consumer relationships between queries and mutations, allowing the generation of realistic chained requests. GraphQLer demonstrates the benefit of dependency reasoning for GraphQL security testing and serves as the strongest baseline in our comparison.

Table 2: Technical specifications of LLMs used for GraphQL testing.

Model	Developer	Year	Size	Focus
`LLaMA 3.1`	Meta	2024	8B	Open-source, efficient inference
`Gemini 2.5 Flash`	Google DeepMind	2025	Undisclosed	cost-efficient reasoning
`GPT-5 Mini`	OpenAI	2025	Undisclosed	optimized for speed
`DeepSeek R1`	DeepSeek AI	2025	671B	Reasoning and efficiency

*https://github.com/SLL288/prediql

Model Selection. For evaluation, we selected four LLMs spanning different developers, sizes, and design philosophies, as summarized in Table 2. Specifically, we included `LLaMA 3.1` from Meta [10] as a representative open source model, `Gemini 2.5` from Google DeepMind [5] as a lightweight and optimized variant of the Gemini family, GPT-5 Mini from OpenAI [6] as a smaller yet efficient version of the GPT-5 series, and `DeepSeek R1` from DeepSeek AI [3], an open source model that emphasizes reasoning and efficiency. This diverse selection allows us to cover both open source and proprietary approaches, models optimized for speed as well as those focused on reasoning capabilities.

Bandit Configuration Details. For experimental evaluation, we instantiated eight distinct bandit arms, each representing a specific prompting strategy defined by four parameters: `Schema`, `Arg Mode`, `Depth`, and `Top-k`. Table 3 summarizes these configurations.

Each configuration corresponds to a balance between reliability and exploration. Conservative arms (*e.g.*, `schema_min_known`) prioritize syntactic validity by reusing known parameter values and shallow nesting. Aggressive arms (*e.g.*, `schema_deep_real`) promote deeper traversal and the synthesis of new argument values. The non-schema variants probe the LLM's ability to generalize without explicit schema guidance. The bandit dynamically reallocates probability mass toward arms that maximize successful, coverage-expanding executions.

Evaluation Metric. To address RQ1 and RQ2, we define the following coverage metric to evaluate GraphQL API testing:

Table 3: Parameters defining adaptive arms in PrediQL's prompt generation.

Arm Name	Schema	Arg Mode	Depth	Top-k
`schema_min_known`	True	known	1	3
`schema_min_real`	True	real	1	3
`schema_mod_known`	True	known	2	5
`noschema_min_known`	False	known	1	3
`noschema_min_real`	False	real	1	0
`schema_min_nulls`	True	nulls	1	3
`schema_deep_known`	True	known	3	5
`schema_deep_real`	True	real	3	5